
第 1 章

Numpy 及其基本使用

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

第 1 章•NumPy 及其基本使用（新版讲义）

本页是第 1 章的章首页（索引）。正文位于 01-数组基础.md ~ 06-常见误区与技巧.md，与原版资料完全分离（原版见 ../../原始资料/）。所有正文代码均已由 build/validate_code.py 自动运行核验，运行结果与文中的 text 输出一致。

本章概览

NumPy 是 Python 科学计算的地基：它用高效的 ndarray（N 维数组）统一表示向量、矩阵和张量，并用 C 语言实现底层计算，速度远快于纯 Python 列表。本章将带你从“创建一个数组”走到“解线性方程组、做矩阵分解、拟合多项式”，为后续 SymPy、SciPy、Pandas、Matplotlib、NetworkX、Statsmodels、scikit-learn 各章打下基础。

学习目标

学完本章，你应该能够：

- 1. 创建并操作 NumPy 数组，理解 ndarray 的属性（shape、ndim、dtype、size）。
- 2. 熟练使用索引、切片、花式索引、布尔掩码，并理解视图 vs 拷贝的区别。
- 3. 掌握广播（broadcasting）规则，能判断两个形状是否可广播。
- 4. 完成向量/矩阵的算术、逻辑、统计运算，正确区分 * 与 @。
- 5. 使用 numpy.linalg 求解线性方程组、计算范数/秩/行列式，进行特征值分解、SVD、Cholesky、QR 分解。
- 6. 用 poly1d / polyfit 做多项式运算与最小二乘拟合，并能用训练/测试误差识别过拟合。
- 7. 用随机数生成器（default_rng）生成可复现的随机数组。
- 8. 完成两个综合案例（SVD 低秩近似、多项式拟合与过拟合），体会 NumPy 在真实数据上的用法。

案例索引

案例卡	所在节	技术点	难度
1. 学生成绩表——创建、切片、花式索引与视图陷阱	01 数组基础	创建/属性/切片/花式索引/布尔/视图拷贝	★☆☆
2. 对每列做 z-score 标准化——广播	01 数组基础	axis 统计、广播	★☆☆
3. 蒙特卡洛估算圆周率——向量化随机数	01 数组基础	default_rng、布尔掩码、向量化	★★☆
4. 传感器滑动平均——滑窗视图与向量化	02 数组运算	sliding_window_view、axis 统计	★★☆
5. 成绩排名与补考改分——argsort / where	02 数组运算	argsort、where、排名	★★☆
6. 成绩表另存与回读——文本与二进制	02 数组运算	savetxt/loadtxt、savez_compressed	★☆☆
7. 解线性方程组——投入产出/电路问题	03 线性代数	solve、det、方程建模	★★☆
8. 最小二乘拟合——身高预测体重	03 线性代数	lstsq、设计矩阵	★★☆
9. 噪声数据的“多项式拟合 + 训练/测试误差”	04 多项式与拟合	polyfit、RMSE、过拟合	★★★

案例卡	所在节	技术点	难度
10. SVD 低秩近似——合成图像压缩	05 综合案例	SVD、低秩近似、重构误差	★★★
11. 模拟实验数据的最小二乘拟合	05 综合案例	训练/测试误差、过拟合	★★★

先修要求与运行环境

- 熟悉 Python 基础语法（变量、列表、函数、循环；可先学 [聪明办法学 Python](#)）。
- 安装 Python 3.10+ 与 NumPy:

```
pip install numpy matplotlib
# 若需要跑上机/作业，再安装
pip install scipy pandas seaborn
```

- 推荐在 JupyterLab / VS Code 中打开 .ipynb 练习；命令行列出的脚本直接保存为 .py 运行即可。

本章目录

小节	文件	内容
01 数组基础	01-数组基础.md	数据类型、创建、属性、索引切片、视图拷贝、变形堆叠、广播、随机数、3 个案例卡
02 数组运算	02-数组运算.md	向量运算、逻辑筛选、矩阵运算、统计与排序、文件存取、3 个案例卡
03 线性代数	03-线性代数.md	逆/秩/行列式/范数、解方程组、最小二乘、特征值分解与 SVD、2 个案例卡
04 多项式与拟合	04-多项式与拟合.md	poly1d、求导/积分、求根、polyfit 拟合、过拟合案例卡
05 综合案例	05-综合案例.md	SVD 低秩近似、模拟实验数据最小二乘拟合（含过拟合，图文案例）
06 常见误区与技巧	06-常见误区与技巧.md	易错点清单、性能技巧、调试建议

数学预备与附录

本章的线性代数/矩阵背景已集中到附录《数学与算法补充》:

- **A 线性代数与矩阵**: A.1 直觉、A.2 定义速查、A.3 分解与最小二乘、A.4 Python 对应;
- **F 通用速查**: F.4 数值稳定性（条件数/灾难消去）;
- [打开附录 A](#) | [附录索引](#)

进入 03 线性代数与 05 综合案例前，先读 A.1~A.3（约 10 分钟）。

练习与上机入口

- [本章练习 \(exercises/\)](#) : 作业题（20 题）、自测 quiz、参考答案。
- [本章上机 \(lab/\)](#) : 循序渐进的上机 notebook，含综合任务与检查清单。
- [本章参考与延伸阅读 \(references.md\)](#) : 官方文档、精品教程、习题集、中文资料。

- [教学说明 \(teaching.md\)](#) : 课时安排、重点难点、考核建议 (教师用)。

建议课时

环节	学时	对应内容
讲课	3-4 学时	01-04 正文 + 05 案例讲解
上机	2-3 学时	lab/ 逐题完成; 课后完成 exercises/ 作业

使用说明

- **学生**: 先读 01-04 正文并运行代码 → 完成 lab 上机 → 提交 exercises 作业 → 自测 quiz 检验。生词/公式不懂时, 到 references.md 找官方对应章节。
- **教师**: 按 teaching.md 的课时表讲; lab 可作为上机课内容; exercises 中的 quiz 带自动评分, 可直接回收。

01 数组基础：创建、属性、索引、广播与随机数

本节目标:把 Python 列表升级为 NumPy 数组,并掌握创建、属性、索引切片、视图/拷贝、广播与随机数。
配套图:images/reshape.png、images/broadcast.png(由 build/make_chap1_figures.py 生成)。

本节目标

- 理解 ndarray 是什么，以及它与 Python 列表的区别；
- 会创建常见数组 (array、zeros、ones、full、arange、linspace、diag、eye)；
- 掌握 shape、ndim、dtype、size、itemsize 等属性；
- 掌握索引、切片、花式索引、布尔掩码；
- 理解**视图 (view) 与拷贝 (copy)**，知道 reshape ravel flatten copy 的区别；
- 掌握广播规则，能手工判断形状是否兼容；
- 使用新版随机数生成器 Generator (default_rng) 生成可复现随机数。

先修

- Python 列表、元组、循环、函数；了解变量类型的含义。

1.1 为什么用 NumPy

Python 列表方便但“慢”，且不能直接做向量化运算。NumPy 的 ndarray 由 C 实现，元素类型统一，支持逐元素运算与广播，这正是科学计算需要的能力。

特性	Python list	NumPy ndarray
元素类型	可混合	必须统一 (dtype)
逐元素运算	需循环/列表推导	直接 $a + b$
内存	对象指针数组	连续内存块
速度	慢	快 (向量化)

1.2 创建数组

1.2.1 从列表/元组创建

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

x = np.array([1, 2, 3, 4, 5])           # 一维数组
y = np.array([[1, 2, 3], [4, 5, 6]])   # 二维数组 (矩阵)
print(x, x.shape, x.dtype)
print(y, y.shape, y.ndim)
print(type(x))

[1 2 3 4 5] (5,) int64
[[1 2 3]]
```

```
[4 5 6]] (2, 3) 2
<class 'numpy.ndarray'>
```

注意：子数组长度必须一致；`np.array([[1,2],[3]])` 会报错或产生 object 数组。

1.2.2 指定数据类型

```
x = np.array([1, 2, 3], dtype=np.int32)
print(x.dtype, x.itemsize) # int32 4
```

```
x2 = x.astype(np.float64)
print(x2.dtype, x2.itemsize) # float64 8
```

```
lst = x.tolist() # 或 list(x)
arr = np.array(lst)
```

内存开销：`astype` 默认返回新数组（拷贝）；如仅查看数值但不改，可避免不必要的转换。

1.2.3 快速创建

```
np.zeros((2, 3)) # 全 0, 形状 (2,3)
np.ones((2, 3)) # 全 1
np.full((2, 3), 42) # 全 42
np.arange(0, 11, 2) # [ 0  2  4  6  8 10]
np.linspace(0, 10, 5) # [ 0.  2.5  5.  7.5 10. ]
np.diag([1, 2, 3]) # 对角线矩阵
np.eye(3) # 3x3 单位矩阵
np.eye(3, 4) # 3x4, 主对角线为 1
np.eye(3, k=1) # 主对角线上方一条对角线为 1

print(np.arange(5)) # [0 1 2 3 4]
print(np.linspace(0, 1, 3)) # [0.  0.5 1. ]
print(np.diag([1, 2, 3]))

[0 1 2 3 4]
[0.  0.5 1. ]
[[1 0 0]
 [0 2 0]
 [0 0 3]]
```

提醒：`zeros/ones/full` 的形状参数是**元组**（两个括号）；`arange` 类似 `range`，终点是开区间。

1.2.4 数组属性

```
a = np.arange(24).reshape(2, 3, 4)
print("shape:", a.shape) # 各维长度
print("ndim :", a.ndim) # 维数 3
print("size :", a.size) # 元素总数 24
print("dtype:", a.dtype) # 数据类型
print("itemsize:", a.itemsize) # 单元素字节数
```

1.3 变形、转置与拼接

1.3.1 reshape

```
arr = np.arange(6)
print(arr.reshape((2, 3))) # 或 arr.reshape(2, 3)
print(arr.reshape((3, -1))) # -1 自动推断: 3×2
print(arr.reshape(-1))      # 展平为一维
```

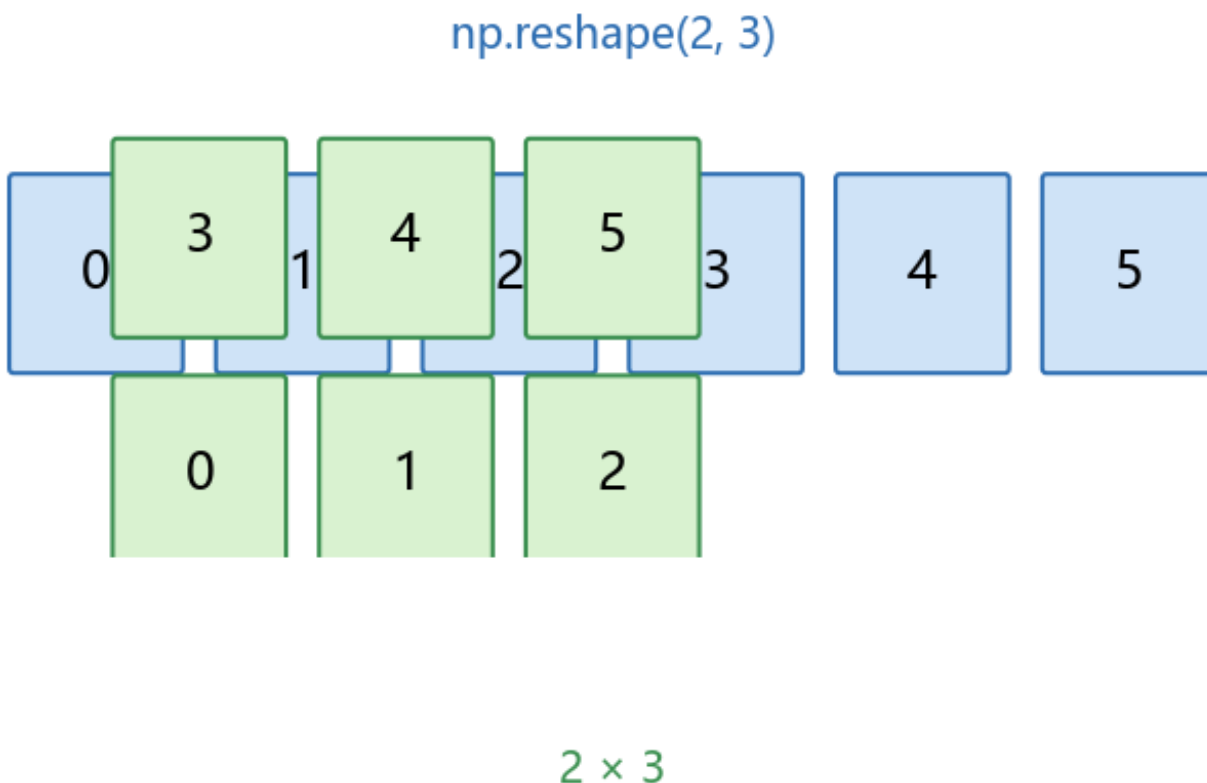


Figure 1: reshape 示意图

- 总元素数必须相等;
- reshape 在内存连续时通常返回视图 (共享内存); 需要独立副本时用 `arr.reshape(...).copy()`。

1.3.2 展平: ravel vs flatten

```
a = np.array([[1, 2, 3], [4, 5, 6]])
a.ravel()      # 视图 (尽可能不拷贝)
a.flatten()    # 总是拷贝
a.reshape(-1)  # 展平
```

1.3.3 转置

```
a = np.arange(6).reshape(2, 3)
a.T      # 转置 (3,2)
```

```
np.transpose(a)           # 等价
np.transpose(a, axes=(1, 0)) # 高维时可指定轴的顺序
```

1.3.4 拼接与堆叠

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6]])
np.concatenate((a, b), axis=0)    # 沿 0 轴拼接: (3,2)
np.concatenate((a, a), axis=1)    # 沿 1 轴拼接: (2,4)
np.vstack((a, b))                 # 垂直堆叠
np.hstack((a, a))                 # 水平堆叠
np.stack((a, b), axis=0)          # 新增维度堆叠 (2,2)
np.column_stack((a, b))           # 一维数组按列堆叠 (2,2)
```

1.3.5 增删元素

```
a = np.array([1, 2, 3])
np.append(a, 4)                   # 返回新数组 [1 2 3 4]
np.delete(a, 1)                   # 返回新数组 [1 3]
a[[0, 2]]                         # 花式索引取第 0、2 个
a[a != 3]                         # 布尔掩码过滤
```

频繁 append/delete 会反复拷贝, 性能差; 大数据请先想好形状或用列表收集。

1.4 索引与切片

1.4.1 基本索引与切片 (视图)

```
a = np.arange(10)
a[0]           # 标量
a[2:5]         # 视图, 切片
a[::-1]        # 反转
```

二维数组:

```
m = np.arange(12).reshape(3, 4)
m[0, 1]       # 第 0 行第 1 列
m[:, 1]       # 第 1 列 (视图)
m[1:3, 1:3]   # 子矩阵 (视图)
```

1.4.2 花式索引 (fancy indexing, 拷贝)

```
m = np.arange(12).reshape(3, 4)
print(m[[0, 2], [1, 3]])    # 取 (0,1) 和 (2,3) → [1 11]
print(m[np.ix_([0, 2], [1, 3])]) # 行 × 列交叉 → 2×2
[ 1 11]
[[ 1  3]
 [ 9 11]]
```


1.4.3 布尔掩码

```
x = np.array([1, -2, 3, -4, 5])
mask = x > 0
print(mask)           # [ True False  True False  True]
print(x[mask])        # [1 3 5]
print(np.where(x > 0, x, 0))  # 正数保留, 否则 0
```

布尔索引返回拷贝; 条件组合用 &、|、~ 且必须加括号, 例如 `x[(x > 0) & (x < 4)]`。

1.5 视图与拷贝（重点）

```
a = np.arange(6)
b = a.reshape(2, 3)      # 通常为视图
b[0, 0] = 99
print(a[0])              # 99 ——共享内存!

c = a.copy()             # 独立拷贝
c[0] = -1
print(a[0])              # 仍为 99

print(a.base is None)    # 原始数组 base 为 None
print(b.base is a)       # True → b 是 a 的视图
```

经验法则:

操作	是否共享内存
基本切片	视图
reshape (连续内存)	视图 (否则拷贝)
ravel	视图 (尽可能)
flatten	拷贝
花式索引 / 布尔索引	拷贝
copy()	拷贝

1.6 广播机制 (broadcasting)

规则: 从最后一个维度往前比较——(1) 两维相等; (2) 或其中一维为 1 (可“拉伸”); (3) 或该维不存在 (自动补 1)。都不满足则报错。

```
A = np.array([[1, 2, 3], [4, 5, 6]])  # (2,3)
b = np.array([10, 20, 30])           # (3,) → 看作 (1,3)
print(A + b)

[[11 22 33]
 [14 25 36]]
```

- $(2,3) + (3,) \checkmark$, 结果为 $(2,3)$;

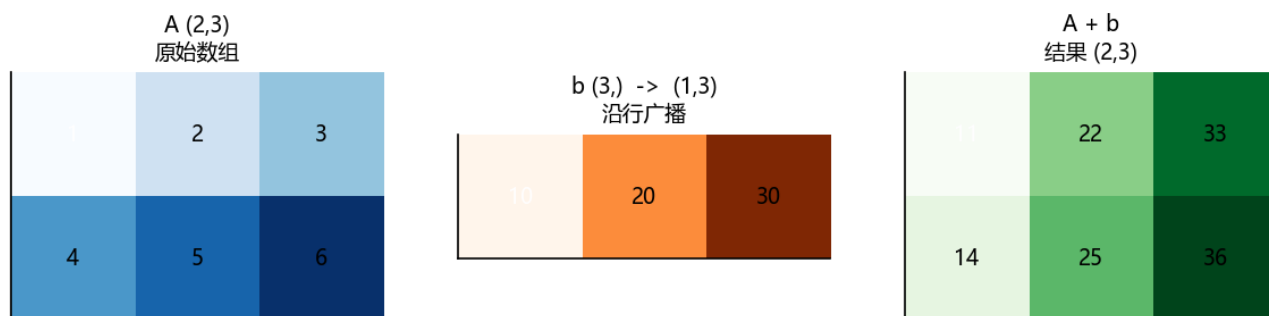


Figure 2: 广播示意图

- $(3,) + (4,) \times$ (末尾不等且无 1);
- $(4,1) + (1,3) \checkmark$, 结果 $(4,3)$;
- $(4,1,6) + (3,6) \checkmark$, 结果 $(4,3,6)$ (把 $(3,6)$ 补成 $(1,3,6)$)。

调试技巧：报错时打印两个数组的 shape，用 `np.newaxis`（即 `None`）手动补维。

1.7 随机数生成与抽样

1.7.1 新版推荐：Generator

```
rng = np.random.default_rng(42)      # 固定种子，结果可复现
rng.random((2, 3))                   # [0,1) 均匀
rng.integers(1, 10, size=5)           # 整数 [1,10)
rng.uniform(-1, 1, size=4)            # 均匀分布
rng.normal(2, 0.5, size=6)           # 正态  $N(2, 0.5^2)$ 
```

1.7.2 旧接口（仍可用，但不建议用于新代码）

```
np.random.seed(42)
np.random.rand(2, 3)                 # [0,1)
np.random.randint(1, 10)             # 整数
np.random.randn(5)                   # 标准正态
np.random.normal(2, 0.5, 5)
```

选型：新代码优先 `default_rng`（更快、线程安全、接口清晰）。

案例卡 1：学生成绩表——创建、切片、花式索引与视图陷阱

目标

把一张“学生 × 科目”成绩表放入 `ndarray`，用切片、花式索引、布尔筛选快速完成任务，并观察视图与拷贝的区别。

代码

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

# 4 名学生, 3 门课 (语文 / 数学 / 英语)
scores = np.array([
    [85, 92, 78],
    [67, 73, 60],
    [95, 88, 90],
    [72, 80, 83],
])

print(" 形状:", scores.shape)           # (4, 3)
print(" 第 2 名学生:", scores[1])       # 行索引 1
print(" 第 1、3 名学生:", scores[[0, 2]]) # 花式索引: 拷贝
print(" 数学成绩:", scores[:, 1])        # 列切片
print(" 数学 < 75 的学生:", scores[scores[:, 1] < 75])

# 基础切片返回视图: 改视图会改原表
view = scores[0:2, :]
view[0, 0] = 99
print(" 基础切片改动原表 (第 0 行第 0 列) :", scores[0, 0])

# copy() 才是独立副本
copy_ = scores.copy()
copy_[0, 0] = 50
print("copy 后原表不变:", scores[0, 0])

# 花式索引返回拷贝: 改“取出的行”不影响原表
row = scores[[0]]
row[0, 0] = 1
print(" 花式索引是拷贝, 改行不影响原表:", scores[0, 0])
```

运行结果 (已运行核验)

```
形状: (4, 3)
第 2 名学生: [67 73 60]
第 1、3 名学生: [[85 92 78]
 [95 88 90]]
数学成绩: [92 73 88 80]
数学 < 75 的学生: [[67 73 60]]
基础切片改动原表 (第 0 行第 0 列): 99
copy 后原表不变: 99
花式索引是拷贝, 改行不影响原表: 99
```

讲解

1. `scores.shape` 是 (4, 3), 第一维是 “行/学生”, 第二维是 “列/科目”。

- 2. `scores[1]` 取第 2 名学生（索引从 0 开始），`scores[:, 1]` 取“数学”整列。
- 3. `scores[[0, 2]]` 用整数列表做**花式索引**：返回**拷贝**，所以之后改取出的行不会影响原表。
- 4. `scores[scores[:, 1] < 75]` 是**布尔掩码**：先算每行“数学 < 75”的真假，再留下对应行。
- 5. **最易踩坑**：`scores[0:2, :]` 是**基础切片**，返回视图；`view[0, 0] = 99` 会直接把原表改成 99。要独立副本显式 `.copy()`。

主要用法 / API

需求	写法
取第 i 行	<code>arr[i]</code>
取第 j 列	<code>arr[:, j]</code>
取指定行/列组合	花式索引 <code>arr[[i1, i2], [j1, j2]]</code> ；交叉组合用 <code>np.ix_</code>
条件筛选	<code>arr[mask]</code> ；多条件 <code>arr[(cond1) & (cond2)]</code>
独立副本	<code>arr.copy()</code> （或 <code>arr[].copy()</code> ）
判断是否共享内存	<code>arr.base is not None</code> 或 <code>np.may_share_memory(a, b)</code>

常见错误

- 用 `and/or` 连接数组条件 → 必须 `&/|` 且加括号；
- 以为切片是拷贝，导致“改了一下原数据全变了”；
- 把 `len(arr)` 当维数：二维数组 `len` 只给第一维长度，用 `arr.ndim/arr.shape`。

拓展

- 给成绩低于 60 的所有课程置 0（答案：`scores[scores < 60] = 0`）；
- 用 `np.argsort` 给三位学生按“数学”排名。

案例卡 2：对每列做 z-score 标准化——广播

目标

对成绩矩阵做“每列独立标准化”： $z = (x - \text{列均值}) / \text{列标准差}$ ，体会广播：均值/标准差是长度为 3 的向量，自动与 (3,3) 矩阵对齐。

代码

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

data = np.array([
    [90.0, 80.0, 70.0],
    [70.0, 60.0, 50.0],
    [50.0, 40.0, 30.0],
])
```

```
mean = data.mean(axis=0) # 每列均值: axis=0 表示“沿第 0 轴折叠”
std = data.std(axis=0)   # 每列标准差
z = (data - mean) / std  # 广播: (3,3) - (3,) → (3,3)

print(" 均值:", mean)
print(" 标准差:", std)
print(" 标准化后:")
print(z)
```

运行结果（已运行核验）

```
均值: [70. 60. 50.]
标准差: [16.3299 16.3299 16.3299]
标准化后:
[[ 1.2247  1.2247  1.2247]
 [ 0.         0.         0.        ]
 [-1.2247 -1.2247 -1.2247]]
```

讲解

- `data.mean(axis=0)` 返回 3 个数（每列一个）；`axis=0` 记住口诀：“消除第 0 维，保留第 1 维”，所以结果是每列的结果。
- `(data - mean)` 中 `mean` 形状为 `(3,)`，广播时被当作 `(1,3)`，自动拉伸到 `(3,3)`。
- `z-score` 后每列均值 ≈ 0 、标准差 ≈ 1 ，后续做 PCA、聚类、回归前常做这一步。

主要用法 / API

- `np.mean(a, axis=k)`、`np.std(a, axis=k)`;
- `(a - mean) / std` 就是向量化标准化;
- 若想返回“整矩阵展平后的均值”，不要传 `axis`。

常见错误

- `axis=0` 和 `axis=1` 记反 → 用“消除哪一维”记忆;
- 使用 `data.std()`（不带 `axis`）得到单一标准差，不是每列标准差;
- 忘记 `np.set_printoptions` 导致输出小数位过多，书本/作业里建议在开头设置一次。

拓展

- 对“每个学生独立标准化”（每行）应如何改？（提示：`axis=1`）
 - 把标准差为 0 的列单独处理，避免除以 0。
-

案例卡 3：蒙特卡洛估算圆周率——向量化随机数

目标

在单位正方形内随机撒点，落在单位圆内的比例 $\approx \pi/4$ 。用 `default_rng` + 向量化布尔掩码，百万个点也可以瞬间算完。

代码

```
import numpy as np

rng = np.random.default_rng(42)  # 固定种子，结果可复现
n = 1_000_000
pts = rng.random((n, 2))         # n 个 (x, y), x,y in [0,1)

inside = (pts[:, 0] ** 2 + pts[:, 1] ** 2) <= 1  # 点到原点距离 ≤ 1
pi_est = 4 * inside.mean()           # 面积比: 圆面积 / 正方形面积 = pi/4

print("inside 比例:", inside.mean())
print("pi 估计:", pi_est)
print("pi 真值:", np.pi)
```

运行结果（已运行核验）

```
inside 比例: 0.785745
pi 估计: 3.14298
pi 真值: 3.141592653589793
```

讲解

- `rng.random((n, 2))` 生成 n 行 2 列；`[:, 0]` 是 x 坐标、`[:, 1]` 是 y 坐标。
- `inside` 是布尔数组：每个点“在圆内”为 `True`，`inside.mean()` 即比例。
- 随机数要**固定种子**才能复现同一结果，课堂演示、作业批改都建议这么做。

主要用法 / API

- `np.random.default_rng(seed)` → 固定种子；
- `rng.random((n, 2))` → 均匀分布 $[0, 1)$ ；
- `rng.integers` / `rng.normal` / `rng.uniform` / `rng.choice` 等。

常见错误

- 用旧接口 `np.random.seed` + `np.random.rand`（新版不推荐，但仍可用）；
- 不设种子 → 每次结果不同，作业无法复现；
- 把“圆内”写成 `< 1` 而不是 `<= 1`（对连续点无影响，但边界点语义不同）。

拓展

- 撒 100 万点是否一定比 1 万点更接近 π ？试试不同 n ；
- 用 3D 蒙特卡洛估算球体积，验证“高维时样本需求激增”。

常见误区

1. *** 不等于矩阵乘法**: $A * B$ 是逐元素乘法; 矩阵乘法用 $A @ B$ 或 `np.matmul`。
2. **忘记 zeros/ones/full 要传元组**: `np.zeros(2, 3)` 会报错, 应为 `np.zeros((2, 3))`。
3. **以为 reshape 一定返回拷贝**: 连续内存下是视图; 改视图会影响原数组。
4. **误用 len(a)**: 一维数组返回长度, 二维返回行数; 看形状用 `a.shape`。
5. **广播不通用**: $(3,) + (4,)$ 不会自动 “截断对齐”。
6. **随机数不设种子**: 不设种子结果每次不同, 课堂演示/作业请用 `default_rng(seed)`。

思考题

1. `np.arange(10)[::2]` 与 `np.arange(10).reshape(5,2)[:,0]` 是否共享内存? 用 `.base` 验证。
2. 解释 `np.ix_` 与同时传入两个顺序索引的区别。
3. 举一个形状满足广播规则却 “符合直觉但结果错误” 的例子 (提示: $(3,1)$ 与 $(3,)$)。

动手练习 (详见 lab)

- 创建 3×4 矩阵, 用布尔掩码把负数置 0;
- 用 `np.arange` 生成 0–100 的偶数, 转成 10×5 矩阵;
- 复现 “对每列做不同缩放” 的广播例子;
- 用 `default_rng(0)` 生成 100 个标准正态数并统计均值/方差。

延伸阅读

- NumPy 官方绝对入门: [链接](#)
- NumPy 官方广播机制: [链接](#)
- SciPy Lecture Notes –NumPy: [链接](#)

02 数组运算：向量、矩阵、逻辑筛选、统计与文件存取

本节目标：熟练进行逐元素运算、点积/叉积、逻辑筛选、矩阵运算、按轴统计与结果存取。配套图：
images/vector_ops.png。

本节目标

- 区分逐元素运算与线性代数运算 (* vs @ / dot);
- 会用布尔掩码与 np.where 做筛选与替换;
- 掌握 sum/mean/std/min/max 沿 axis 方向统计;
- 会用 sort/argsort 排序与排名;
- 会用 savetxt/loadtxt、save/load、savez_compressed 保存与读取数组。

先修

- 01 数组基础（创建、索引、广播）。
-

2.1 向量的运算

2.1.1 逐元素操作

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

v = np.array([2, 3])
w = np.array([3, 1])
print(v + w)      # [5 4]
print(v - w)      # [-1 2]
print(2 * v)       # [4 6]
print(v * w)       # 逐元素乘 [6 3]
print(v / w)       # 逐元素除 [0.6667 3.    ]
print(v ** 2)      # 逐元素平方 [4 9]
```

2.1.2 数量积（点积）与叉积

```
a = np.array([1, 2])
b = np.array([3, 4])
print(np.dot(a, b))      # 1*3 + 2*4 = 11
print(a @ b)             # 向量点积同样可用 @

c = np.array([1, 2, 3])
d = np.array([4, 5, 6])
print(np.cross(c, d))     # [-3  6 -3]
```

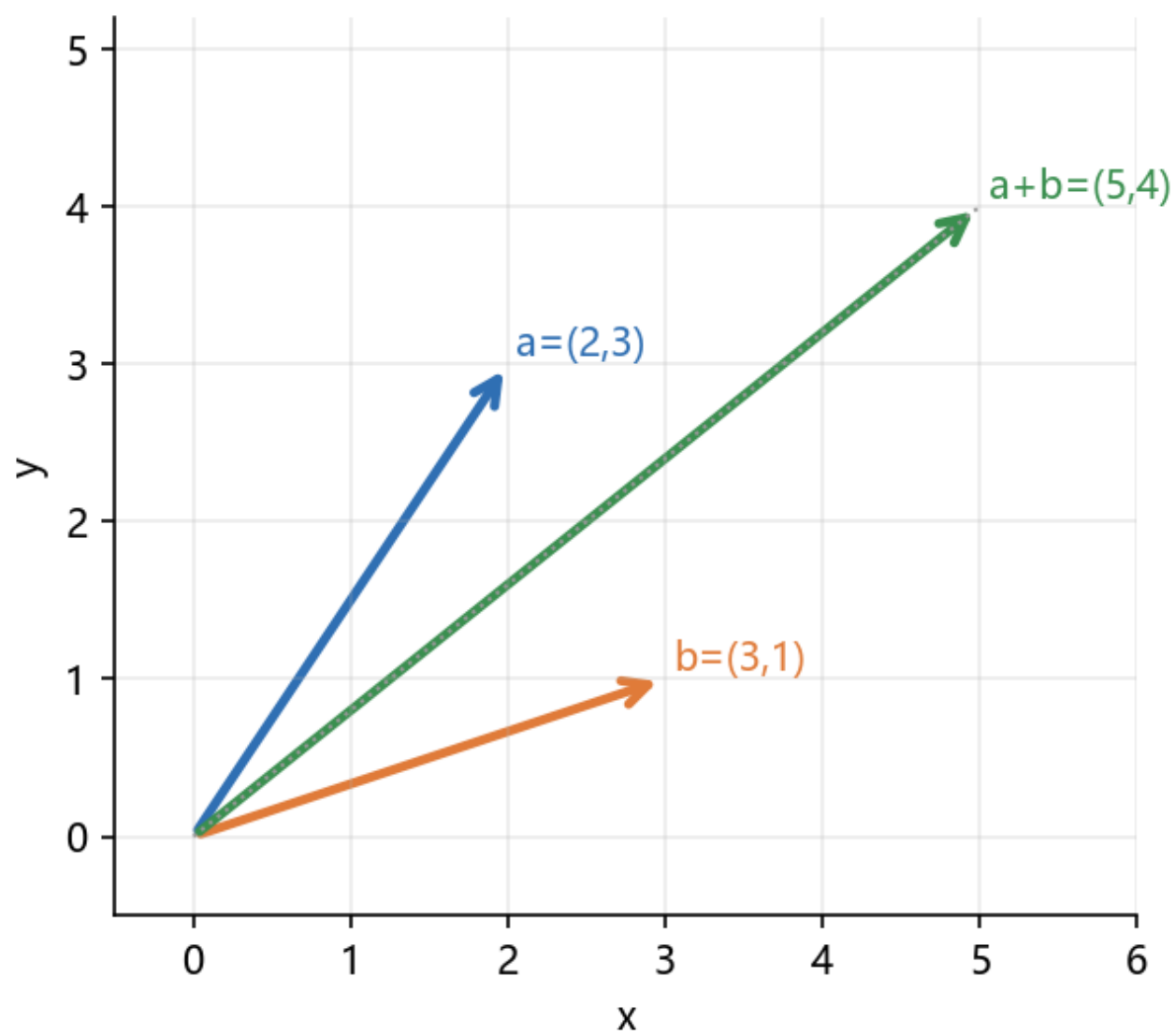



Figure 1: 向量运算示意

2.1.3 向量的范数

```
x = np.array([3, 4])
np.linalg.norm(x)          # 5.0 (默认 2-范数)
np.linalg.norm(x, ord=1)   # 7.0 (1-范数)
```

2.1.4 排序

```
a = np.array([3, 1, 2])
np.sort(a)          # 返回新数组 [1 2 3]
a.sort()            # 原地排序
np.argsort(a)       # 排序后原索引
```

2.2 向量/矩阵的逻辑运算——筛选

```
x = np.array([1, -2, 3, -4, 5])

print(x[x > 0])          # [1 3 5]
print(x[(x > 1) & (x < 5)]) # [3]
print(np.where(x > 0, x, 0)) # [1 0 3 0 5]
```

二维按行筛选：

```
m = np.array([[1, 9], [6, 2], [7, 8]])
print(m[(m[:, 0] > 3) & (m[:, 1] < 9)]) # [[6 2] [7 8]]
```

组合条件必须加括号，且不能用 and / or，要用 & / | / ~。

2.3 矩阵的运算

2.3.1 算术运算

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print(A + B)      # 矩阵加法
print(A - B)      # 矩阵减法
print(A * B)      # 注意：逐元素乘
print(A @ B)      # ✓ 矩阵乘
print(np.matmul(A, B)) # 同 @
```

为什么 * 不是矩阵乘：NumPy 中 * 是 ufunc 的逐元素运算；要数学上的矩阵乘法，用 @（或 np.dot / np.matmul，二者在 2D 下等价，但 @ 语义更明确）。

2.3.2 矩阵上的统计

```
S = np.array([[85, 90, 80],
              [78, 85, 90],
              [90, 95, 85]])
```

```

S.sum(axis=0)      # 每列和 [253 270 255] (三门课总分)
S.sum(axis=1)      # 每行和 [255 253 270] (每个学生总分)
S.mean(axis=0)     # 每列均值
S.max(axis=1)      # 每行最大值
S.std()            # 全体标准差

```

记忆: axis=0 表示“沿第 0 轴 (行方向) 折叠”, 即求**每列**的统计量; axis=1 求**每行**。

2.3.3 查找 (where / argmax)

```

m = np.array([[3, 7], [9, 2]])
np.where(m > 5)      # (array([0,1]), array([1,0])) 坐标
np.argmax(m)         # 展平后的最大索引 2 → (1,0)
np.argmax(m, axis=1) # 每行最大索引 [1 0]

```

2.4 结果的保存与读取

```

a = np.array([[1, 2], [3, 4]])

np.savetxt("result.txt", a, fmt="%.2f", delimiter=",")
b = np.loadtxt("result.txt", delimiter=",")

np.save("a.npy", a)      # 二进制 (推荐: 更快、保留 dtype)
c = np.load("a.npy")

np.savez_compressed("data.npz", x=a, y=b)
d = np.load("data.npz")
d["x"], d["y"]

```

文本格式 savetxt 直观但慢、且可能丢精度; 大数组优先二进制 np.save/np.savez_compressed。

案例卡 1: 传感器滑动平均——滑窗视图与向量化

目标

对一串传感器读数做“窗口大小为 3”的移动平均, 用 `numpy.lib.stride_tricks.sliding_window_view` 一步生成全部窗口, 避免显式 for 循环。

代码

```

import numpy as np
np.set_printoptions(precision=4, suppress=True)

x = np.array([1.0, 3.0, 2.0, 5.0, 4.0, 6.0, 2.0])

# sliding_window_view 返回“视图”(不复制数据), 形状 = 原长 - 窗口 + 1

```

```
w = np.lib.stride_tricks.sliding_window_view(x, 3)
print(" 窗口矩阵:")
print(w)
print(" 移动平均:", w.mean(axis=1))
```

运行结果（已运行核验）

窗口矩阵:

```
[[1. 3. 2.]
 [3. 2. 5.]
 [2. 5. 4.]
 [5. 4. 6.]
 [4. 6. 2.]]
```

```
移动平均: [2.          3.3333  3.6667  5.          4.          ]
```

讲解

- `sliding_window_view(x, 3)` 得到 (5, 3) 的窗口矩阵: 第 i 行 = $x[i:i+3]$ 。
- `w.mean(axis=1)` 对每行（每个窗口）求均值，即移动平均。
- 这是视图，不复制数据；适合窗口大、数据长的场景。

主要用法 / API

- `numpy.lib.stride_tricks.sliding_window_view(a, window_size)`;
- 配合 `mean/std/diff` 等做平滑、差分、局部统计;
- 需要“截断首尾”时用 `np.convolve` 或 `pandas.Series.rolling`。

常见错误

- 忘记这是视图，修改窗口会改原数组;
- 把它当“卷积”：窗口求均值与卷积核为 $1/w$ 的卷积在边界处理上不同;
- 对 `mean(axis=1)` 写成 `axis=0`，得到每列统计。

拓展

- 改成“指数加权移动平均”（提示：`np.convolve` + 权重向量）;
- 对二维图像做 3×3 局部均值滤波（提示：两层滑窗）。

案例卡 2：成绩排名与补考改分——`argsort` / `where`

目标

给一组成绩排名（从高到低），并把不及格（ < 62 ）的学生分数改为 65。核心是 `argsort` 的“索引排序”和 `np.where` 的条件替换。

代码

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

scores = np.array([85, 92, 78, 60, 95, 88])

# argsort(-scores): 分数取负后从小到大排, 等价于原分数从大到小的索引
order = np.argsort(-scores)

# 为每位学生计算名次 (1 表示最高分)
ranks = np.empty(6, dtype=int)
ranks[order] = np.arange(1, 7)

print(" 从高到低:", scores[order])
print(" 每位学生的排名 (对应原位置):", ranks)

# 条件替换: < 62 的分数补考改成 65
fixed = np.where(scores < 62, 65, scores)
print("60 分补考为 65 后:", fixed)
```

运行结果 (已运行核验)

```
从高到低: [95 92 88 85 78 60]
每位学生的排名 (对应原位置): [4 2 5 6 1 3]
60 分补考为 65 后: [85 92 78 65 95 88]
```

讲解

- `np.argsort(-scores)` 返回 “排序后的原索引”: `order[0]` 是最高分所在位置。
- `scores[order]` 按 `order` 重新取值, 就得到从高到低的成绩序列。
- 排名时: `ranks[order] = np.arange(1, 7)`, 意思是 “第 1 名放在 `order[0]` 的位置”, 这是 NumPy 里 “按位置反填” 的常用技巧。
- `np.where(cond, a, b)` 等价于 “条件成立取 `a`, 否则取 `b`”, 但对数组**逐元素**执行。

主要用法 / API

- `np.argsort(a)`: 升序索引; `np.argsort(-a)`: 降序索引;
- `np.where(cond, x, y)`: 条件替换/筛选;
- `np.sort(a)` 返回新数组, `a.sort()` 原地排序;
- 简化排名可用 `scipy.stats.rankdata` 或 `pandas.Series.rank`。

常见错误

- 混淆 `np.sort` 与 `np.argsort`: 前者给排序后的值, 后者给排序后的索引;
- 用 `scores[scores < 62] = 65` 也能改分, 但 `np.where` 写法更显式、且不改变原数组;
- 排名时正序/逆序写反。

拓展

- 若有并列分数，如何让并列者名次相同（提示：研究 `scipy.stats.rankdata` 的 `method`）；
- 按“数学成绩”排名后只输出前 3 名学生名单。

案例卡 3：成绩表另存与回读——文本与二进制

目标

把学生成绩矩阵保存成 CSV（文本）与 `.npz`（二进制压缩），再读回来，比较两种方式。

代码

```
import numpy as np

data = np.array([[85, 92, 78], [67, 73, 60]], dtype=int)

# 文本: CSV, 可被 Excel 打开
np.savetxt("_numpy_scores.csv", data, fmt="%d", delimiter=",")
loaded = np.loadtxt("_numpy_scores.csv", delimiter=",")

# 二进制: 更快、保留 dtype, 且可压缩多个数组
np.savez_compressed("_numpy_scores.npz", data=data)
d = np.load("_numpy_scores.npz")

print("loadtxt:")
print(loaded)
print("npz data:")
print(d["data"])
```

运行结果（已运行核验）

```
loadtxt:
[[85. 92. 78.]
 [67. 73. 60.]]
npz data:
[[85 92 78]
 [67 73 60]]
```

讲解

- `np.savetxt` 输出文本，`fmt="%d"` 保证整数格式；`loadtxt` 读回后默认转成 `float64`。
- `np.savez_compressed` 把多个数组打进一个 `.npz`，读回用 `d["data"]`；保留 `dtype`，且体积更小、速度更快。
- 实际项目中：**中间结果用二进制 npz 快存快取；给 Excel/同学看的导出 CSV。**

主要用法 / API

- `np.savetxt(path, a, fmt=..., delimiter=...)` / `np.loadtxt(path, delimiter=...)`;

- `np.save / np.load` (单数组.npy);
- `np.savez_compressed / np.load` (多数组.npz)。

常见错误

- `savetxt` 对高精度 float 默认保留 `%.18e`, 导致文件很大且不可读——指定 `fmt`;
- `loadtxt` 读文本遇到表头会报错, 表头用 `skiprows=1`;
- 用 `np.savez_compressed` 后忘记用 `d["key"]` 取出, 直接 `print(d)` 只能看到键。

拓展

- 把 CSV 读成“带表头 + 中文列名”的表格, 应改用 `pandas.read_csv` (第 4 章);
- 逐列比较二进制与文本读回的精度差异。

常见误区

1. *** 与 @ 混用**: `A * B` 是逐元素乘; 数学矩阵乘必须 `@`。
2. **axis 方向记反**: `sum(axis=0)` 是“消除第 0 维”, 得到每列结果。
3. **条件用 and/or**: 数组布尔运算必须 `&/|`, 且每个条件加括号。
4. **反复 np.append**: 每次产生新数组, 大数组循环中极慢; 先估算容量或用列表。
5. **np.dot 语义在多维下容易混淆**: 明确用 `@` 更直观。
6. **用 np.save 存文本**: 二进制文件没有扩展名也可以 `np.load`, 但 CSV 用 `savetxt` 再 `loadtxt`。

思考题

1. `np.dot`、`np.matmul`、`@` 在三维数组上行为是否一致? 查文档说明。
2. 为什么 `np.where(cond, x, y)` 要求 `x`、`y` 可广播到 `cond` 的形状?
3. 用 `argsort` 给三位学生按总分排名。

动手练习 (详见 lab)

- 生成 5×5 随机矩阵, 把主对角线元素置 0;
- 对每列做 z-score 标准化 $((x - \text{mean}) / \text{std})$, 并比较 `axis` 的选择;
- 用 `savetxt/loadtxt` 存取一份成绩表并检查数值一致。

延伸阅读

- NumPy 官方 ndarray 基础: [链接](#)
- NumPy 官方 I/O 说明: [链接](#)
- SciPy Lecture Notes –数组操作: [链接](#)

03 线性代数：逆、方程组、分解与 SVD

本节目标：用 `numpy.linalg` 做逆/伪逆、秩、行列式、范数、解方程组、最小二乘与常用矩阵分解。
配套图：`images/cond_sensitivity.png`、`images/svd_compression.png`。

本节目标

- 会求逆矩阵、伪逆、秩、行列式、范数；
- 会用 `np.linalg.solve` 解线性方程组，理解条件数 (condition number) 对数值稳定的影响；
- 会用 `np.linalg.lstsq` 做最小二乘；
- 掌握特征值分解、SVD、Cholesky、QR 分解的基本用法与适用场景；
- 能说出 SVD 的“压缩/降维”思想 (见 05 综合案例)。

先修

- 01 数组基础、02 数组运算；线性代数的基本概念 (矩阵、行列式、特征值)。

3.1 基本线性代数运算

3.1.1 逆矩阵与伪逆

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)
```

```
A = np.array([[3, 1], [1, 2]])
A_inv = np.linalg.inv(A)
print(A_inv)
print(A @ A_inv)      # ≈ 单位矩阵
```

非方阵或奇异矩阵用 `pinv` (伪逆)，常用于最小二乘与超定方程组：

```
B = np.array([[1, 2], [2, 4], [3, 6]])  # 秩亏
print(np.linalg.pinv(B).shape)          # (2, 3)
```

3.1.2 秩

```
np.linalg.matrix_rank(np.array([[1, 2], [2, 4]]))  # 1 (第二行是第一行的 2 倍)
np.linalg.matrix_rank(np.array([[1, 2], [3, 4]]))  # 2
```

3.1.3 行列式

```
A = np.array([[3, 1], [1, 2]])
print(np.linalg.det(A))  # 5.000000000000001 (浮点近似)
```

行列式为 0 等价于矩阵奇异，此时方程组没有唯一解。

3.1.4 范数

```
v = np.array([3, 4])
np.linalg.norm(v)  # 5.0, 2-范数
```



```

np.linalg.norm(v, ord=1)      # 7.0

M = np.array([[3, 4], [0, 5]])
np.linalg.norm(M)             # Frobenius 范数 ≈ 7.0711
np.linalg.norm(M, ord=2)      # 谱范数 (最大奇异值)
np.linalg.norm(M, ord=np.inf) # 无穷范数 (行和最大值)

```

3.2 线性方程组的求解

3.2.1 直接求解: np.linalg.solve

```

A = np.array([[3, 1], [1, 2]]) # 系数矩阵
b = np.array([9, 8])          # 常数向量
x = np.linalg.solve(A, b)
print(x)                      # [2. 3.]
print(A @ x)                  # ≈ [9. 8.]

```

适用条件: A 是方阵且非奇异 (存在唯一解)。结构更一般的线性方程组用 lstsq。

3.2.2 最小二乘: np.linalg.lstsq

超定 (方程多于未知数) 或欠定 (方程少于未知数) 时, 求 “误差平方和最小” 的解:

```

X = np.array([[1, 1], [1, 2], [1, 3], [1, 4]])
y = np.array([6, 5, 7, 10])
coef, residuals, rank, sv = np.linalg.lstsq(X, y, rcond=None)
print(coef)      # [3.5 1.4] → y ≈ 3.5 + 1.4 x

```

3.2.3 条件数与数值稳定性

条件数 $\text{cond}(A)$ 刻画方程组对抗扰动的敏感程度: 条件数越大, 解越 “脆弱”。

```

A = np.array([[1, 2, 3],
               [2, 4.0001, 6],
               [1, 0.9999, 2]])
b = np.array([1, 2, 3])
print(np.linalg.cond(A)) # 这个矩阵近似奇异, cond 很大
x0 = np.linalg.solve(A, b)

```

结论: 即使 b 的扰动非常小 ($1e-4$ 量级), 解的变化也可能很大; 工程中应尽量避免病态方程组, 或换用更稳定的分解方法。

3.3 矩阵分解

3.3.1 特征值分解 (eig)

只适用于 **方阵** (且通常要求可对角化):

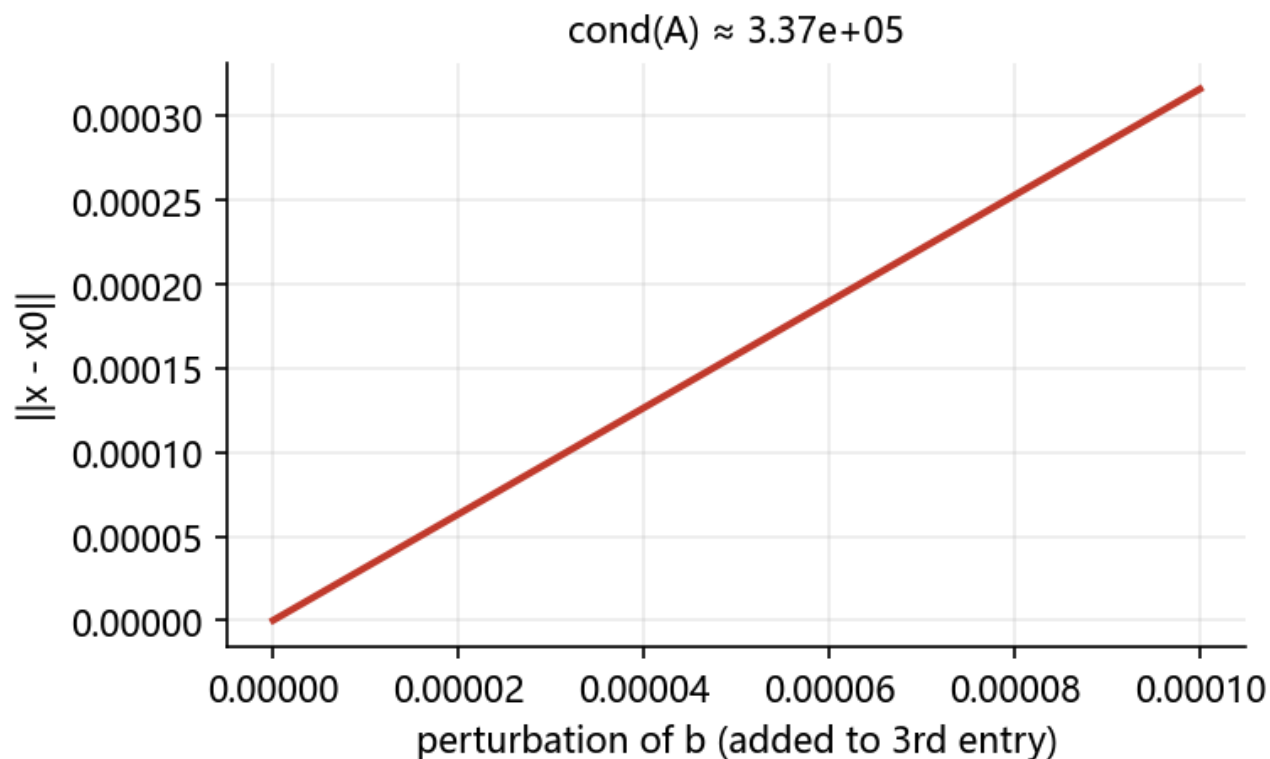


Figure 1: 条件数敏感性

```
A = np.array([[2, 1], [1, 2]])
w, v = np.linalg.eig(A)
print(w)           # [3. 1.] 特征值
print(v)           # 列向量为对应特征向量
print(A @ v[:, 0] == w[0] * v[:, 0]) # 验证  $Av = \lambda v$  (近似相等)
```

3.3.2 奇异值分解 (SVD)

对任意矩阵 (包括非方阵) 适用:

```
M = np.array([[1, 2], [3, 4], [5, 6]], dtype=float)
U, S, Vt = np.linalg.svd(M, full_matrices=False)
print(U.shape, S.shape, Vt.shape) # (3,2) (2,) (2,2)
print(S)                          # 奇异值 (降序)
print(U @ np.diag(S) @ Vt)        # 重构  $\approx M$ 
```

为什么重要: 奇异值从大到小排列, 只保留前 k 个奇异值即可得到“最优低秩近似”, 这是数据压缩、降维 (PCA)、图像压缩的基础。见 [05 综合案例](#)。

3.3.3 Cholesky 分解

要求矩阵**对称正定**:

```
A = np.array([[4., 2.], [2., 3.]])
L = np.linalg.cholesky(A)
```

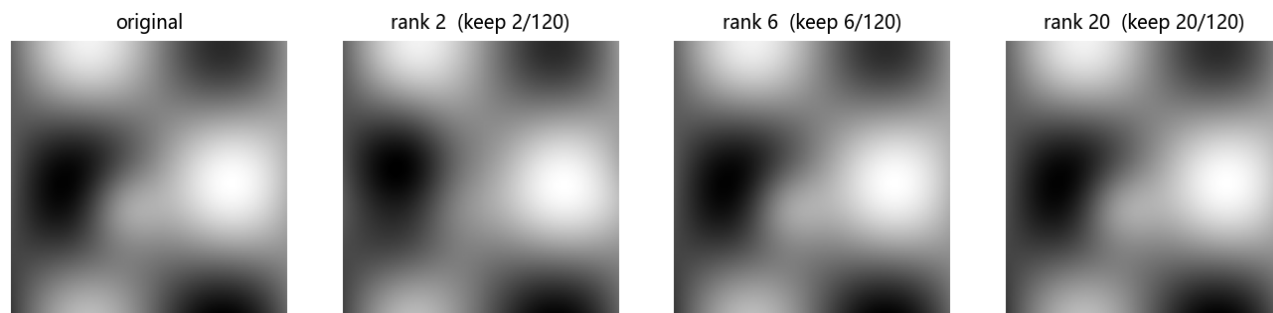


Figure 2: SVD 压缩示例

```
print(L)          # [[2. 0. ][1. 1.4142]]
print(L @ L.T)    # ≈ A
```

3.3.4 QR 分解

```
A = np.array([[12., -51.], [6., 167.], [-4., 24.]])
Q, R = np.linalg.qr(A)
print(Q.shape, R.shape)  # (3,2) (2,2)
print(np.allclose(Q @ R, A))  # True
```

QR 在最小二乘、特征值算法稳定实现中广泛使用。

案例卡 1：解线性方程组——投入产出/电路问题

目标

求解下面的二元一次方程组（可看成“两个部门投入产出平衡”或“两环路电流方程”）：

$$\begin{cases} 2x + y = 5 \\ x + 3y = 10 \end{cases}$$

用 `np.linalg.solve` 一次性求出 (x, y) ，并验证 $Ax = b$ 。

代码

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

A = np.array([[2.0, 1.0],
              [1.0, 3.0]])
b = np.array([5.0, 10.0])

x = np.linalg.solve(A, b)
print(" 解:", x)          # x=[1, 3]
```

```
print(" 验证 Ax:", A @ x)
print("det:", np.linalg.det(A))    # 近似 5
```

运行结果（已运行核验）

```
解: [1. 3.]
验证 Ax: [ 5. 10.]
det: 5.000000000000001
```

讲解

- 把方程组写成矩阵形式 $Ax = b$: A 是系数矩阵, b 是常数向量, 解 $x = A^{-1}b$ 。
- `np.linalg.solve` 内部用 LU 分解, 比直接 `inv(A) @ b` 更稳定、更快。
- `det(A)=5\neq0`, 说明有唯一解; 若 `det` 接近 0, 说明系数矩阵病态 (见 3.2.3)。

主要用法 / API

- `np.linalg.solve(A, b)`: 方阵且非奇异时解唯一;
- `np.linalg.inv(A)`: 求逆 (一般不要直接用 `inv@b` 解方程);
- `np.linalg.det(A)`: 行列式;
- `np.linalg.matrix_rank(A)`: 秩。

常见错误

- 方程个数与未知数个数不相等 → 用 `lstsq` 而不是 `solve`;
- `b` 写成行向量 `[[5,10]]` 而不是列向量 `[5,10]` (1D 会自动处理, 但 2D 会报错);
- 忽略条件数, 认为 `solve` 返回成功就一定可靠。

拓展

- 若把第 2 个方程改成与第 1 个线性相关, 观察 `solve` 报错与 `det=0`;
- 把该问题推广到 3 个未知数, 并解释 “三个平面交于一点”。

案例卡 2: 最小二乘拟合——身高预测体重

目标

用一组 (身高, 体重) 数据拟合线性关系 $y = ax + b$ (x 身高, y 体重), 用 `np.linalg.lstsq` 求出 (a, b) , 并与 `np.polyfit` 对照。

代码

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

rng = np.random.default_rng(7)
height = np.linspace(150, 190, 20)           # 20 个身高样本
weight_true = 0.8 * height - 70             # 真实线性关系
```

```
weight = weight_true + rng.normal(0, 3, height.size) # 加测量噪声

# 设计矩阵: 第 1 列全是 1 (截距项), 第 2 列是身高
X = np.column_stack([np.ones(height.size), height])
coef, res, rank_, sv = np.linalg.lstsq(X, weight, rcond=None)

print("coef (截距, 斜率):", f"{coef[0]:.4f}, {coef[1]:.4f}")
print(" 残差平方和:", f"{float(res[0]):.4f}")
print("rank:", rank_)
print(" 来自 np.polyfit 的对照:", np.round(np.polyfit(height, weight, 1), 4))
```

运行结果 (已运行核验)

```
coef (截距, 斜率): -60.2551, 0.7371
残差平方和: 92.6892
rank: 2
来自 np.polyfit 的对照: [ 0.7371 -60.2551]
```

讲解

- 预测模型: $y \approx 0.7371x - 60.2551$, 即身高每高 1 cm, 体重平均约增加 0.737 kg。
- `lstsq` 返回 4 个值: 系数、残差平方和、矩阵秩、奇异值; `rcond=None` 使用默认截断阈值。
- `np.polyfit(height, weight, 1)` 与 `lstsq` 结果一致 (系数顺序不同: `polyfit` 是降幂, 所以先斜率后截距)。

主要用法 / API

- `np.linalg.lstsq(X, y, rcond=None)`;
- `np.polyfit(x, y, deg)` 是一元多项式最小二乘 (本质也是 `lstsq`);
- 残差越小、且系数在“生物学/物理上说得通”, 拟合才可信。

常见错误

- 忘记在 X 中加“全 1 列” → 截距项被强制为 0;
- 用 1D 线性回归却把 X 写成 `(n,)` 而非 `(n,1)`;
- 只看训练集残差, 不看测试集 (见 04 章过拟合)。

拓展

- 加入“年龄”作为第二特征, 做多元回归 (X 加一列);
- 用 `np.linalg.svd` 手写求解 `lstsq`, 理解最小二乘与伪逆的关系。

常见误区

1. 对非方阵用 `inv`: 应使用 `pinv` 或 `lstsq`。
2. 认为 `eig` 对任意矩阵都可用: 特征值分解只对方阵; 非方阵用 SVD。
3. 忽略条件数: `solve` 计算成功 $\neq$ 解可靠, 先看 `cond`。
4. SVD 返回的 S 不是对角矩阵: 需要 `np.diag(S)` 构造。

5. `full_matrices=False` 与非默认的区别：不关心零奇异值/补全正交基时，用 `full_matrices=False` 更省内存。
6. 直接 `np.linalg.inv(A) @ b`：数值上不如 `np.linalg.solve(A, b)` 稳定，且多一次矩阵求逆。

思考题

1. 为什么 `cond(A)` 很大时，`np.linalg.solve(A, b)` 的结果可能仍“看起来正常”？
2. SVD 的 `U`、`Vt` 与 `eig` 的特征向量有什么联系（提示： $A^T A$ ）？
3. 若只保留前 1 个奇异值做近似，误差如何估计？

动手练习（详见 lab）

- 解一个 4×4 线性方程组 3 组（含病态）并比较解；
- 对 10×5 矩阵做 SVD，比较 rank 1/3/5 重构误差；
- 用 Cholesky 验证正定矩阵的分解唯一性。

延伸阅读

- NumPy 官方 `linalg` 参考：[链接](#)
- NumPy 官方广播与 `linalg` 教程：[链接](#)
- SciPy Lecture Notes – 线性代数：[链接](#)

04 多项式与拟合：poly1d、求根与最小二乘拟合

本节目标：用 `np.poly1d` 构造/运算多项式，会求导、积分、求根；用 `np.polyfit` 做最小二乘拟合，并理解过拟合。配套图：[images/polyfit.png](#)。

本节目标

- 会用 `np.poly1d` 构造、求值、加减乘除多项式；
- 会求导 (`deriv`)、积分 (`integ`)、求根 (`np.roots`)；
- 会用 `np.polyfit` / `np.polyval` 做最小二乘拟合，并理解“过拟合”现象；
- 知道新版 `np.polynomial.Polynomial` 的方向（便于迁移）。

先修

- 01 数组基础、02 数组运算；了解“多项式系数按最高次到常数排列”。

4.1 多项式的构造与运算

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)
```

```
# 2x^2 + 3x + 1
p = np.poly1d([2, 3, 1])
print(p)           # 2 x^2 + 3 x + 1
print(p(5))        # 66
```

4.1.1 四则运算

```
p = np.poly1d([2, 3, 1])      # 2x^2+3x+1
q = np.poly1d([1, -1])        # x - 1

print(p + q)                  # 加法
print(p - q)                  # 减法
print(p * q)                  # 乘法 (系数卷积)
quotient, remainder = p / q   # poly1d 直接除返回 (商, 余数) 元组
print(quotient)
print(remainder)
```

4.1.2 求导与积分

```
p = np.poly1d([2, 3, 1])
print(p.deriv())              # 4x + 3
print(p.deriv(2))             # 二阶导
print(p.integ())              # 0.6667x^3 + 1.5x^2 + x (常数默认为 0)
print(p.integ(m=2))           # 二重积分
```

4.1.3 系数与求根

```
p = np.poly1d([1, -3, 2])      #  $x^2 - 3x + 2$ 
print(p.coeffs)                # [ 1. -3.  2.]
print(p.c)                     # 按降幂的系数
np.roots(p)                    # [2.  1.]
```

`np.roots` 也支持直接传系数数组: `np.roots([1, -3, 2])`。

4.2 最小二乘拟合: `polyfit`

```
x = np.array([0, 1, 2, 3, 4], dtype=float)
y = np.array([1, 2, 9, 28, 65], dtype=float)  # 恰好是  $x^3 + 1$ 

coef = np.polyfit(x, y, 3)      # 3 次多项式最小二乘
print(np.round(coef, 4))        # [ 1.  0. -0.  1.] (浮点误差约  $1e-14$ )
p = np.poly1d(coef)
print("p(2.5) =", round(np.polyval(coef, 2.5), 4))  # 16.625
```

含噪声数据的拟合与过拟合:

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(7)
def f(x): return 2 * x ** 2 - 3 * x + 1
x = np.linspace(-2, 3, 40)
y = f(x) + np.random.normal(0, 2, x.size)

for deg in (1, 3, 5):
    coef = np.polyfit(x, y, deg)
    print(deg, ":", np.round(coef, 4))

xs = np.linspace(-2, 3, 200)
plt.scatter(x, y, s=16, label="data")
for deg, c, ls in [(1, "grey", "--"), (3, "orange", "-."), (5, "green", "-")]:
    plt.plot(xs, np.polyval(np.polyfit(x, y, deg), xs), c=c, ls=ls, label=f"deg {deg}")
plt.legend(); plt.savefig("poly_fit_demo.png")
```

观察: 次数越高, 拟合曲线越可能贴近噪声 (过拟合); 次数过低则拟合不足。选择次数可用交叉验证或残差分析, 而不是盲目升高次数。

4.3 新版多项式接口 (了解)

NumPy 1.x 之后的 `np.polynomial` 提供更现代、更稳定的 `Polynomial` / `Chebyshev` 等类:

```
from numpy.polynomial import Polynomial
p = Polynomial([1, 3, 2])      # 注意: 系数从常数项开始!
```

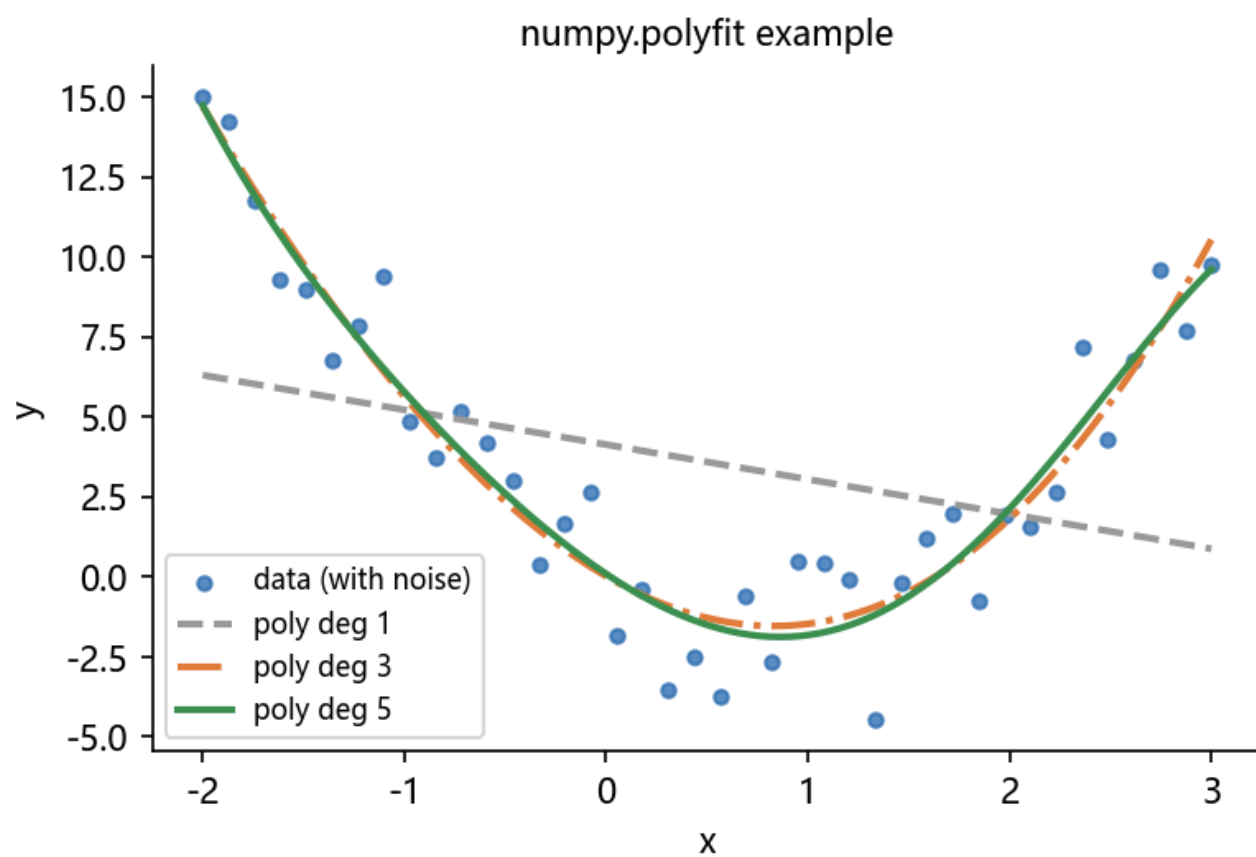



Figure 1: polyfit 示例

```
print(p(2))                # 1 + 3*2 + 2*2^2 = 15
print(p.roots())
```

`np.poly1d` 是旧接口（系数降幂、基于 `np.poly`），大量教程/教材仍在使⤵用；新代码可优先考虑 `np.polynomial`。本讲义两者都给出，以兼容常见场景。

案例卡 1：噪声数据的“多项式拟合 + 训练/测试误差”——一次、二次、五次对比

目标

对“已知二次函数 + 正态噪声”的数据分别用 1、2、5 次多项式拟合，比较训练集与独立测试集的 RMSE，直观看到**次数越高不代表越好**。

代码

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

rng = np.random.default_rng(7)
def f(x): return 2 * x ** 2 - 3 * x + 1

x = np.linspace(-2, 3, 40)
y = f(x) + rng.normal(0, 2, x.size)

def rmse(coef, xx, yy):
    return np.sqrt(np.mean((np.polyval(coef, xx) - yy) ** 2))

# 全量看一眼系数
for deg in (1, 2, 5):
    print("deg", deg, "coef:", np.round(np.polyfit(x, y, deg), 4))

# 前 28 个点训练, 后 12 个点独立测试
x_train, y_train = x[:28], y[:28]
x_test, y_test = x[28:], y[28:]
for deg in (1, 2, 5):
    c = np.polyfit(x_train, y_train, deg)
    print(f"deg={deg}  train RMSE={rmse(c, x_train, y_train):.3f}  test RMSE={rmse(c, x_test, y_test):.3f}")
```

运行结果（已运行核验）

```
deg 1 coef: [-1.0864  4.1331]
deg 2 coef: [ 2.2266 -3.313  -0.1869]
deg 5 coef: [-0.0503  0.0661  0.4681  1.8231 -4.2196  0.0869]
deg=1  train RMSE=2.395  test RMSE=14.206
deg=2  train RMSE=1.578  test RMSE=2.936
deg=5  train RMSE=1.428  test RMSE=98.279
```

讲解

- **一次**：欠拟合，训练/测试误差都大（模型太简单）；
- **二次**：正好对应当前模型，训练误差略高于五次，但测试误差最小（2.936）——**选对模型**；
- **五次**：训练误差最低（1.428）但测试误差暴增到 98.279——**过拟合**，它在“背噪声”。
- 结论：**选次数必须看独立测试集/交叉验证，而不是训练误差。**

主要用法 / API

- `np.polyfit(x, y, deg)`：最小二乘多项式系数（降幂）；
- `np.polyval(coef, x)`：用系数在 x 处求值；
- `np.poly1d(coef)`：包装成多项式对象，支持 `deriv()/integ()/roots()`；
- `np.polynomial.Polynomial`：新版接口（系数常数项在前）。

常见错误

- 系数顺序搞反：`poly1d([a, b, c])` 表示 $ax^2 + bx + c$ ；`Polynomial` 是常数项在前；
- 用“训练集 RMSE 最小”选模型 → 必然选最高次，必须用测试集/交叉验证；
- 数据点少于或等于次数时，多项式会“精确穿过所有点”（插值），此时训练误差为 0 但毫无预测能力。

拓展

- 用 `np.linalg.lstsq` 重写上面的拟合，体会 `polyfit` 只是它的特例；
- 用 `sklearn.model_selection.train_test_split` 随机划分训练/测试集，重复 10 次看 RMSE 分布；
- 给 5 次拟合加 ℓ_2 正则（岭回归），观察测试误差如何变化。

常见误区

1. **系数顺序搞反**：`poly1d([a, b, c])` 表示 $ax^2 + bx + c$ ；`np.polynomial.Polynomial` 则是常数项在前。
2. **polyfit 次数过高**：数据少时高次多项式会震荡/过拟合。
3. **忘记噪声与残差**：拟合不是“解释唯一真理”，要报告残差与预测区间。
4. **把 polyfit 当精确解**：它是数值最小二乘，数据含噪声时系数有误差。
5. **直接用 np.polyval 外推太远**：多项式在训练范围外可能剧烈振荡。

思考题

1. n 个数据点做 $n - 1$ 次多项式拟合会出现什么现象？为什么？
2. `polyfit` 与 `lstsq` 的关系是什么？
3. 如何判断“次数挑对了”？

动手练习（详见 lab）

- 对 $y = 0.5x^3 - 2x + 1 + \varepsilon$ 的 30 个噪声点，分别拟合 1/3/5 次，比较残差与测试误差；
- 用 `poly1d` 求 $x^4 - 5x^2 + 4$ 的根，并验证代入为 0。

延伸阅读

- NumPy 官方 polynomial 文档: [链接](#)
- NumPy 官方 polyfit 说明: [链接](#)
- SciPy Lecture Notes –最小二乘: [链接](#)

05 综合案例：SVD 低秩近似与最小二乘拟合（含过拟合）

本章新增综合案例，把 01–04 的知识串起来：**形状/广播 + 线性代数 + 拟合 + 可视化**。所有代码均可在 NumPy ≥ 1.24 环境直接运行；配图由 build/make_chap1_figures.py 生成。

案例 1：SVD 低秩近似——用 3.3% 的存储量近似一张 120×120 合成图

目标

把一张 120×120 的合成“灰度图”看作矩阵，用 SVD 只保留前 k 个奇异值，观察“压缩率 vs 重构误差”，理解 SVD 是图像压缩、PCA、数据降维的共同原理。

背景

一张 $m \times n$ 的灰度图可看作矩阵。SVD 给出

$$A = U \Sigma V^T, \quad U \in \mathbb{R}^{m \times r}, \Sigma \in \mathbb{R}^{r \times r}, V \in \mathbb{R}^{n \times r}$$

只保留前 k 个奇异值，得到低秩近似

$$A_k = U_{:,1:k} \Sigma_{1:k,1:k} V_{1:k,:}^T$$

存储量从 mn 减少到 $k(m + n + 1)$ ，当 $k \ll \min(m, n)$ 时压缩明显。

代码

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

# 1) 生成一张“合成灰度图” (120x120, 含正弦图案、高斯亮斑与渐变)
x = np.linspace(0, 1, 120); y = np.linspace(0, 1, 120)
X, Y = np.meshgrid(x, y)
img = (np.sin(6 * X) * np.cos(6 * Y)
        + np.exp(-((X - 0.4) ** 2 + (Y - 0.6) ** 2) * 40)
        + 0.6 * X - 0.4 * Y)
img = (img - img.min()) / (img.max() - img.min())

# 2) SVD
U, S, Vt = np.linalg.svd(img, full_matrices=False)

# 3) 不同秩的重建与误差
for k in (2, 6, 20):
    A_k = U[:, :k] @ np.diag(S[:k]) @ Vt[:k, :]
    mse = np.mean((img - A_k) ** 2)
    params = k * (img.shape[0] + 1 + img.shape[1])
    print(f"rank={k:2d}  MSE={mse:.5f}  存储量={params} / 原图={img.size} "
          f"({100*params/img.size:.1f}%)")
```

运行结果 (已运行核验)

```
rank= 2  MSE=0.00087  存储量 =482 / 原图 =14400 (3.3%)
rank= 6  MSE=0.00000  存储量 =1446 / 原图 =14400 (10.0%)
rank=20  MSE=0.00000  存储量 =4820 / 原图 =14400 (33.5%)
```

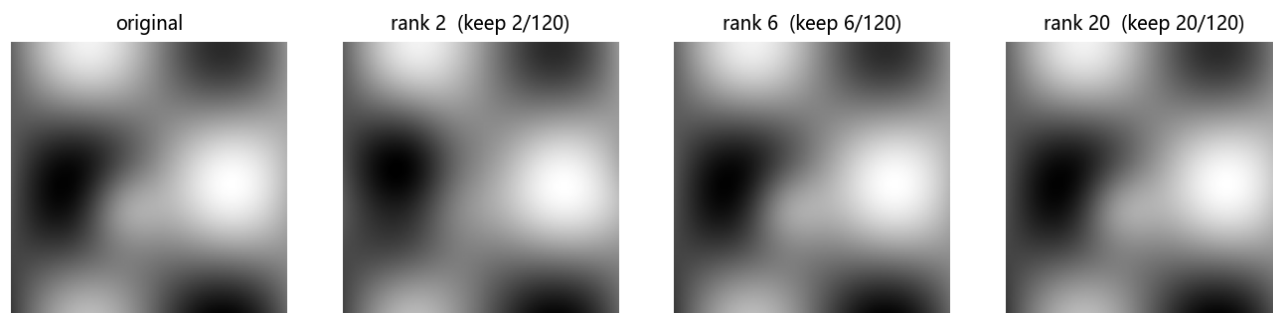


Figure 1: SVD 图像压缩

讲解

- `np.linalg.svd(img, full_matrices=False)` 返回 U (120,120)、 S (120,)、 Vt (120,120); S 是**奇异值向量** (降序), 不是对角矩阵。
- 重构公式 $U[:, :k] @ np.diag(S[:k]) @ Vt[:k, :]$: 只取前 k 个奇异值对应的“列/行”, 就得到最优低秩近似。
- $rank=2$ 时仅用 3.3% 的存储量, $MSE \approx 0.0009$ (肉眼几乎无差别); $rank=6$ 之后误差已低于打印精度。
- 为什么效果好? **自然图像的能量高度集中在前几个奇异值** (低频/整体结构), 后面的奇异值对应高频细节与噪声。

主要用法 / API

- `np.linalg.svd(M, full_matrices=False)`: 经济型 SVD;
- `np.diag(S[:k])`: 把奇异值向量转成对角阵;
- `np.mean((img - A_k) ** 2)`: MSE 重构误差;
- 若要“压缩数量”, 可先保存 $U[:, :k]$ 、 $S[:k]$ 、 $Vt[:k, :]$, 用时再重建。

常见错误

- 直接 $U @ S @ Vt$ 会报错: S 不是对角矩阵;
- 忘记 `full_matrices=False` 可能得到很大的 U/Vt (补全正交基), 浪费内存;
- 只看 MSE 不看视觉: MSE 很低不等于“视觉无损”, 反之亦然;
- 把“压缩率”理解成文件压缩率: 这里的“存储量”是**矩阵元素个数**, 不含编码/位深。

拓展

1. 用 `np.linalg.norm(img - A_k, 'fro')` 计算相对误差;
2. 对一张真实图片 (PIL 读取灰度化) 重复实验, 观察“压缩率 vs 视觉质量”;
3. 把 `img` 换成随机噪声图, 观察奇异值衰减是否还那么快? 为什么?
4. 画出 S 的对数谱 (`plt.plot(np.log10(S))`), 看“能量集中在前几个奇异值”。

案例 2：模拟实验数据的最小二乘拟合——模型正确时能恢复参数，过拟合会在测试集原形毕露

目标

用“已知二次模型 + 高斯噪声”模拟一批实验数据，分别做 2 次与 5 次多项式拟合，比较训练集与独立测试集的 RMSE，体会“模型设定正确时最小二乘能恢复参数，次数越高越容易背噪声”。

背景

物理实验得到一组 (x_i, y_i) ，理论公式是二次函数，但测量有噪声。用 `np.polyfit` 求 $y = ax^2 + bx + c$ 的最佳系数，并评估拟合质量。（为便于课堂复现，这里用模拟数据而非真实仪器读数。）

代码

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

np.random.seed(7)
def theory(x): return 2 * x ** 2 - 3 * x + 1

x = np.linspace(-2, 3, 40)
y = theory(x) + np.random.normal(0, 2, x.size)

def rmse(c, xx, yy):
    return np.sqrt(np.mean((np.polyval(c, xx) - yy) ** 2))

# 前 28 个点训练，后 12 个点独立测试
x_train, y_train = x[:28], y[:28]
x_test, y_test = x[28:], y[28:]

coef2 = np.polyfit(x_train, y_train, 2) # 与理论模型一致
coef5 = np.polyfit(x_train, y_train, 5) # 模型更复杂（可能过拟合）

print("train deg2 coef:", coef2)
print("train deg5 coef:", coef5)
print("RMSE train:", round(rmse(coef2, x_train, y_train), 3),
      round(rmse(coef5, x_train, y_train), 3))
print("RMSE test :", round(rmse(coef2, x_test, y_test), 3),
      round(rmse(coef5, x_test, y_test), 3))
```

运行结果（已运行核验）

```
train deg2 coef: [ 1.8416 -3.4436  0.8602]
train deg5 coef: [-0.9174 -1.5886  1.2521  4.336  -2.8519  0.5595]
RMSE train: 1.967 1.646
RMSE test : 3.111 139.106
```

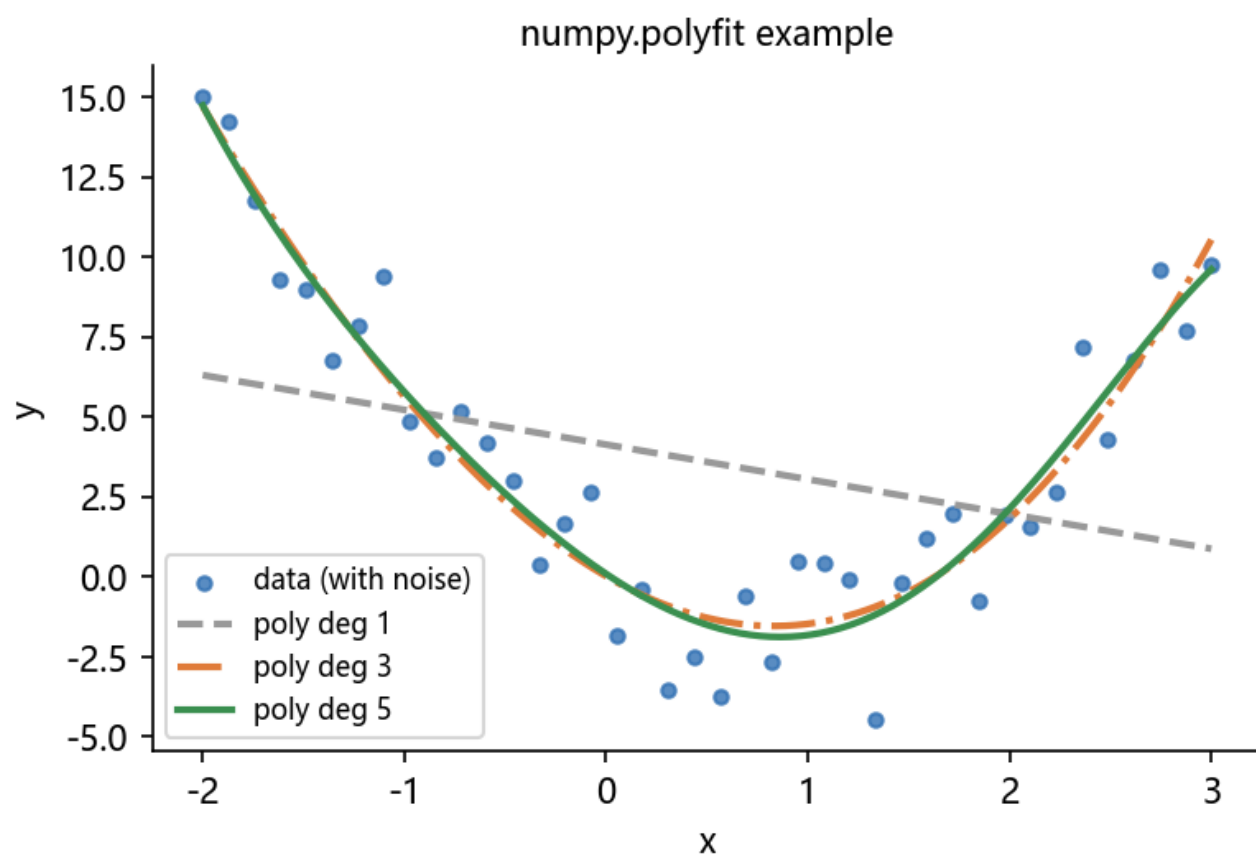


Figure 2: polyfit 示例

讲解

- **二次拟合系数接近真实值** $(2, -3, 1)$ (1.84、-3.44、0.86，噪声不大时能恢复参数)，说明“模型设定正确”时最小二乘有效；
- **五次拟合在训练集上的 RMSE 更低** (1.646 vs 1.967) ——它能更紧地贴合训练数据；
- 但在**独立测试集**上：deg2 RMSE=3.111，而 deg5 RMSE≈139.1！五次多项式在训练范围外/区间间剧烈振荡，过拟合暴露无遗。
- 结论：**选模型/次数必须看独立测试集或交叉验证，而不是训练误差。**

主要用法 / API

- `np.polyfit(x, y, deg)`: 最小二乘多项式系数 (降幂)；
- `np.polyval(coef, x)`: 多项式求值；
- `np.sqrt(np.mean((pred - y) ** 2))`: RMSE (均方根误差)；
- `np.linalg.lstsq`: 更通用的最小二乘 (见 03 章案例 2)。

常见错误

- 用训练 RMSE 选次数 → 一定选高次；
- 测试集与训练集有重叠 / 测试集太小 → 评估不可靠；
- 把“模拟数据”当成真实实验数据汇报 → 应写明数据来源与噪声假设；
- 不做残差分析，直接相信一个“很低”的测试 RMSE。

拓展

1. 划分训练/测试集时使用 `sklearn.model_selection.train_test_split` (随机划分, 代替前 28/后 12)；
2. 用 `np.linalg.lstsq` 重写拟合, 体会与 `polyfit` 的关系；
3. 打印 95% 置信区间 (提示: 用 `np.polyfit` 的 `cov` 参数或 `scipy.stats`)；
4. 增加 ℓ_2 正则 (Ridge) 看测试误差能否下降。

本章综合练习 (上机任务)

1. 生成 50×50 的“数字 + 渐变”图案, 用 $\text{rank} = 1, 3, 5, 10$ 重构并绘制 2×2 对比图；
2. 构造 $y = 3e^{-x} + 2 + \varepsilon$ 的噪声数据, 尝试多项式拟合 (哪种次数更合理?), 并给出误差曲线；
3. 把案例 1 与案例 2 整理成一个 .ipynb, 写一份 200 字以内的结论。

延伸阅读

- NumPy 官方 SVD: [链接](#)
- SciPy 相关低秩近似 (`scipy.sparse.linalg`): [链接](#)
- scikit-learn PCA 官方示例 (下一章会用到): [链接](#)

06 常见误区与技巧

本章为新增“查漏补缺”页：把学生最容易踩的坑集中列出，并给出正确的写法和性能技巧。

1. 高频误区速查表

#	误区 / 错误写法	正确做法	原因
1	A * B 当矩阵乘法	A @ B 或 np.matmul(A, B)	* 是逐元素乘 (ufunc)
2	np.zeros(2, 3)	np.zeros((2, 3))	形状要传元组，两个括号
3	以为 reshape 是拷贝	需要独立数据用 arr.reshape(...).copy()	连续内存的 reshape 是视图
4	x[x > 0 and x < 5]	x[(x > 0) & (x < 5)]	数组布尔运算用 &，且加括号
5	len(a) 猜数组维数	a.shape、a.ndim	len 只给第一维长度
6	对非方阵 np.linalg.inv	np.linalg.pinv / lstsq	逆只定义于方阵
7	np.linalg.svd(M) 直接 U@S@Vt	U @ np.diag(S) @ Vt	S 是奇异值向量，不是对角阵
8	认为 eig 适用于任意矩阵	方阵用 eig，任意矩阵用 svd	eig 数学条件限制
9	np.random.rand 不设种子	default_rng(42)	结果不可复现
10	循环里 np.append	预分配数组或列表收集	每次 append 都会新建数组
11	混淆 axis=0/1	记“消除哪一维就写哪一维”	sum(axis=0) 消行得列统计
12	用 astype 想“改原数组”	arr.astype 返回新数组； 原地改 dtype 不可直接	类型转换是拷贝

2. 原理级难点：视图 vs 拷贝

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)
```

```
a = np.arange(6)
b = a.reshape(2, 3)      # 视图
b[0, 0] = 99
print(a[0])              # 99

c = a.copy()
c[0] = 0                  # 不影响 a
print(a[0])              # 99

判断是否共享内存：
b.base is a              # True → 视图
np.may_share_memory(a, b) # 更稳的检查
```

为什么重要：视图省内存、速度快；但并发修改或函数返回到别人手里时容易“隐形改数据”。对外返回数据前，若不想共享，显式 .copy()。

3. 广播规则速记

1. 从右往左对齐;
2. 相同或一方为 1 $\rightarrow$ 兼容;
3. 全部兼容 $\rightarrow$ 结果为各维最大; 否则 ValueError。

```
import numpy as np

A = np.zeros((4, 1, 6))
B = np.zeros((3, 6))
try:
    print((A + B).shape)      # (4, 3, 6), B 自动补成 (1,3,6)
except ValueError as e:
    print(" 广播失败: ", e)
```

注意: (3,) + (4,) 不兼容; (3,1) + (3,) 也不兼容 (末尾 1 vs 3)。想 “列向量与行向量做外积” 要用 (3,1) + (1,3)。

4. 性能技巧

- **向量化优先**: 能一次 `a[np.newaxis, :] + b[:, np.newaxis]` 就不要双重循环;
- **out= 原地计算**: `np.add(a, b, out=a)` 避免额外分配;
- **in-place 算术**: `a += b` (注意类型安全);
- **einsum 表达张量收缩**: `np.einsum('ik,kj->ij', A, B)` 既是矩阵乘, 也能表达更复杂收缩且减少中间数组;
- **np.lib.stride_tricks.sliding_window_view** 做滑窗 (移动平均/卷积) 时避免显式复制:

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)
```

```
x = np.arange(10)
w = np.lib.stride_tricks.sliding_window_view(x, 3)
print(w.mean(axis=1))    # 窗口大小为 3 的移动平均
```

- **大数据组用 np.savez_compressed** 存储; 文本 savetxt 慢且丢精度。

5. 调试建议

1. 出错先打印 shape / dtype, 再用 `np.allclose` 比较;
2. 用 `np.set_printoptions(precision=4, suppress=True)` 让输出可读;
3. 复杂布尔条件拆成中间变量 (`mask1 = ...; mask2 = ...`);
4. 随机/拟合问题**固定种子** (建议 `default_rng(seed)`), 便于复现与讨论;
5. 用 `np.isnan` / `np.isinf` 检查异常值。

6. 本章小结 (自测)

- ☐ 能解释 dtype 与 itemsize;
- ☐ 能说出 view vs copy 的 4 种情形;
- ☐ 能判断 (2,3)+(3,)、(4,1)+(1,3) 的可广播性;
- ☐ 能区分 *、@、np.dot;
- ☐ 能用 solve / lstsq 解不同形态方程组;

- 能用 `svd` / `polyfit` 完成 05 综合案例。

延伸阅读

- NumPy 官方 “NumPy for MATLAB users”: [链接](#)
- NumPy 性能建议: [链接](#)